

欧洲信息学赛事分析与选讲

NOI2026 冬令营
钟诚

目录

- 引言
- 欧洲信息学赛事一览
- 欧洲信息学赛事的题目风格和组织形式
- 题目选讲
- 小结

引言

- 中国自1984年举办NOI、1989年参加首届IOI以来，在信息学竞赛领域取得了十分优异的成绩。NOI系列活动的OI科技树也枝繁叶茂、蓬勃发展，不断有新的数据结构、算法，以及命题、解题模式涌现。
- 近年来，IOI和其它各级各类国际赛事也进入了高速发展期，产生了许多形式新颖的题目，其中部分类型和考点的题目在NOI系列活动中出现的比重较小。
- 今天的课程旨在抛砖引玉，通过欧洲各大信息学赛事的风格分析和题目选讲，帮助同学们开拓思维、博采众长，了解更多国际比赛的题型特征与解题策略，鼓励大家在未来为 NOI 系列活动添砖加瓦、注入更多活力与创新。

欧洲信息学赛事一览

- 中欧信息学奥林匹克 CEOI
 - 创办于1994年，有8个（从2026年起9个）正式成员国和若干受邀参赛国，是欧洲历史最悠久的历史学赛事之一
- 欧洲女生信息学奥林匹克 EGOI
 - 创办于2021年，近60个国家参与了2025年的比赛，是欧洲参与最广泛的信息学赛事之一
- 欧洲初中生信息学奥林匹克 EJOI
 - 创办于2017年，近30个国家参与了2025年的比赛
- 西欧信息学奥林匹克 WEOI
 - 创办于2023年，有10个固定的国家参与
- 巴尔干半岛信息学奥林匹克 BalkanOI，及其初中生版 JBOI

.....

后面我们在讲到具体题目时，会再详细介绍该赛事的具体开展情况和题目风格。

欧洲信息学赛事的题目风格和组织形式

- 对高级数据结构和算法的要求较低
 - 欧洲赛事更注重思维方面的考察；除个别国家外，各国的国内赛事几乎不超过NOIP的考察范围
 - 即使是在较高级别的赛事中，也有不少不超过CSP-J知识范围的题目，以鼓励更多的同学参与
- 交互题等思维性题目的占比较高
 - 和NOI系列活动的题目相比，欧洲信息学赛事交互题、构造题、思维性题目的比重较高，而对传统数据结构和算法的考察比重相对略低
- 部分赛事有包括笔试、在线考试在内的更多考察形式
 - 受各种硬件、经费等因素限制，许多欧洲国家的国内赛事的前几轮往往以在线的形式，在本校或者家中完成；也有的允许在一定的时间段（如一两个月）内完成，目的是激励大家边学边考

题目选讲

欢迎同学们踊跃发言交流

树上搜索（欧洲初中生信息学奥林匹克 EJOI 2023）

现有一棵 N 个节点的二叉树，根节点为1。树上有一个待寻找的特殊节点。

你能看到二叉树的完整形态，但不知道特殊节点是哪个。你每次可以选取一个节点，询问以该节点为根的子树是否包含特殊节点。

$N \leq 100\,000$ ，最多询问35次。

<https://www.luogu.com.cn/problem/P11354>

树上搜索（欧洲初中生信息学奥林匹克 EJOI 2023）

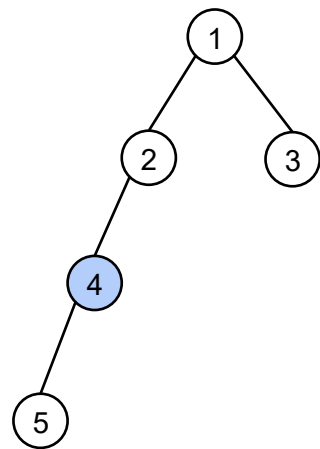
系统输出	用户输出
5 1 1 2 4	
	? 4
1	
	? 5
0	
	! 4

在此样例中，系统首先给定二叉树的规模 $N = 5$ ，以及节点 2~5 的父节点编号：1, 1, 2, 4。

随后，用户询问以 4 为根的子树是否包含特殊节点，系统回答 1，表示“是”。

用户继续询问以 5 为根的子树是否包含特殊节点，系统回答 0，表示“否”。

最后，用户输出猜测的特殊节点编号 4。



树上搜索（欧洲初中生信息学奥林匹克 EJOI 2023）

解法一：由于可以看到二叉树的完整形态，因此可以预处理以每个节点为根的子树的大小（节点个数）。

初始时，特殊节点的可能范围是整棵树。

从根开始，每次找较大的那棵子树，如果该子树超过当前可能范围的 $1/2$ ，那么继续找它的较大的子树，直到某个子树不超过 $1/2$ 。

由于每次找的是较大子树，因此最后找的子树的大小一定不低于可能范围的 $1/4$ 。我们询问该子树是否包含特殊节点后，至少能把可能范围缩小为原先的 $3/4$ 。

因此，总的询问次数至多是 $\log_{\frac{4}{3}} N \approx 40$ 。

树上搜索（欧洲初中生信息学奥林匹克 EJOI 2023）

解法二：与解法一类似，初始时，特殊节点的可能范围是整棵树。

从根开始，每次找较大的那棵子树，如果该子树超过当前可能范围的 $2/3$ ，那么继续找它的较大的子树，直到某个子树不超过 $2/3$ 。

由于每次找的是较大子树，因此最后找的子树的大小一定不低于可能范围的 $1/3$ ，而根据询问的规则，它的大小也不超过可能范围的 $2/3$ 。于是，我们询问该子树是否包含特殊节点后，至少能把可能范围缩小为原先的 $2/3$ 。

因此，总的询问次数至多是 $\log_{\frac{3}{2}} N \approx 28$ 。

中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

一棵二叉树有 n 个节点，分别用正整数 $1, 2, 3, \dots, n$ 表示。它的前序遍历、后序遍历已经确定。中序遍历完全不确定，或只确定其中连续的一段。

求有多少种可能的中序遍历，输出模999 999 937的结果即可。

数据范围： $n \leq 500\,000$

<https://www.luogu.com.cn/problem/P13804>

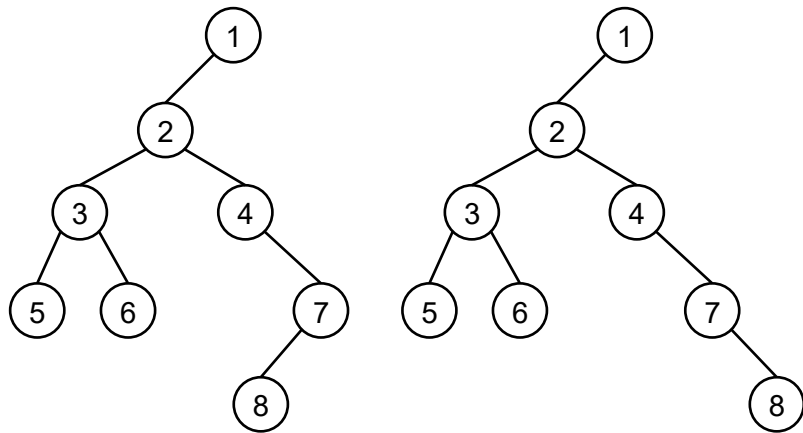
中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

样例输入	样例输出
8 1 2 3 5 6 4 7 8 // 前序遍历 5 6 3 8 7 4 2 1 // 后序遍历 0 0 6 2 4 0 0 0 // 中序遍历	2

在输入的中序遍历中，0表示不确定的节点，正整数表示确定的节点。保证正整数部分是连续的一段。

该样例中，可能的中序遍历有下面2种：

- 5 3 6 2 4 8 7 1
- 5 3 6 2 4 7 8 1



中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

样例输入	样例输出
3 1 2 3 3 2 1 0 0 0	4

该样例中，可能的中序遍历有下面4种：

- 3 2 1
- 2 3 1
- 1 3 2
- 1 2 3

样例输入	样例输出
4 1 2 3 4 4 3 2 1 0 4 0 0	3

该样例中，可能的中序遍历有下面3种：

- 2 4 3 1
- 1 4 3 2
- 3 4 2 1

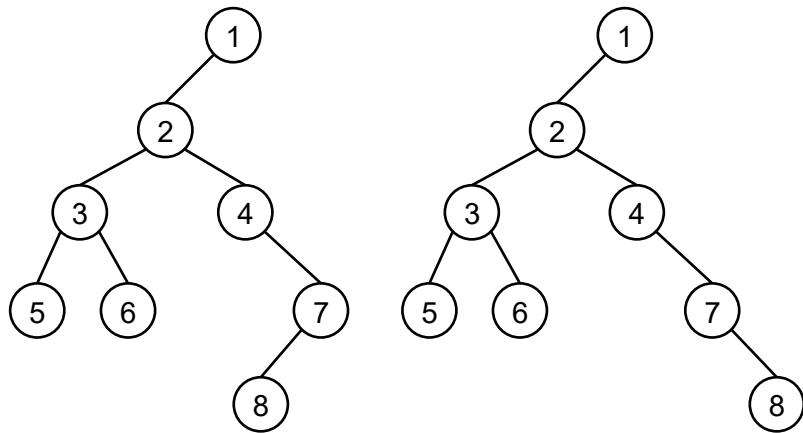
中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

我们一定做过一道经典的题目：已知中序、后序遍历，求前序遍历，这个题目也曾经是NOIP 2001普及组的第3题。

然而，如果仅仅知道前序、后序遍历，是无法唯一确定中序遍历的，参考本题的样例即可验证这一结论：

前序遍历：1 2 3 5 6 4 7 8

后序遍历：5 6 3 8 7 4 2 1



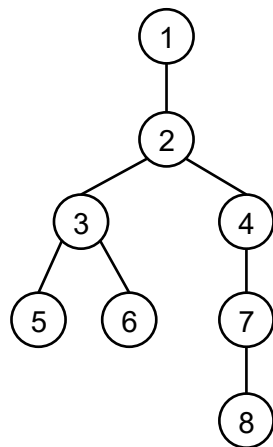
中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

我们设 k 为中序遍历中确定的节点个数，循序渐进解决问题，先考虑 $k = 0$ 的情况。

如果只知道前序、后序，虽然无法唯一确定这棵二叉树，但可以确定到下面这个程度：如果一个节点有两个子节点，那么它的左右子节点可以唯一确定；如果只有一个子节点，那么我们只能确定其父子关系，无法知道是左儿子还是右儿子。

所以，只看样例里的前序、后序，不看中序的话，可以确定到右图这个程度。其中，凡是垂直的竖线，都表示只知道父子关系，但不知道是左儿子还是右儿子。

于是，我们就看有几条“垂直的竖线”，每条都有2种不同的可能，于是就对应着2的次幂种不同的中序遍历。



中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

当 $k = 1$, 且根是中序遍历中确定的节点时。

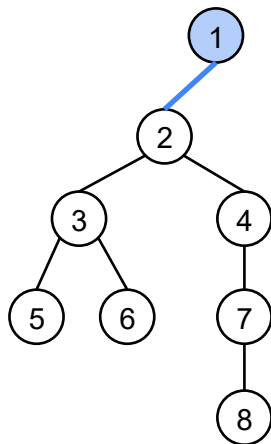
唯一确定的节点就是根, 意味着根如果只有一个子节点的话, 我们可以知道它是左儿子还是右儿子了。其它与 $k = 0$ 的情况相同。

例如:

前序遍历: 1 2 3 5 6 4 7 8

后序遍历: 5 6 3 8 7 4 2 1

中序遍历: 0 0 0 0 0 0 0 1



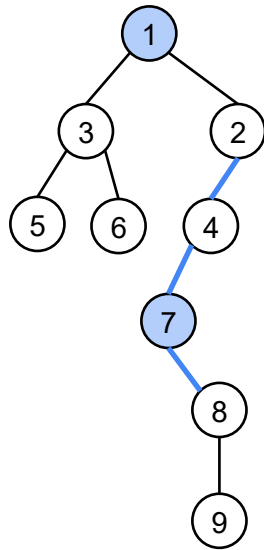
中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

当 $k = 2$ ，且根是中序遍历中确定的节点时。

这时候，除了根以外，还有一个节点 x 也确定了，比如是中序遍历中根的后一个点，那也就是根的右子树里的点。根据中序遍历的性质可知：

- 对于节点 x 的所有祖先（除了根以外），如果只有一个子节点，那么一定是左子节点
- 对于节点 x ，如果只有一个子节点，那么一定是右子节点

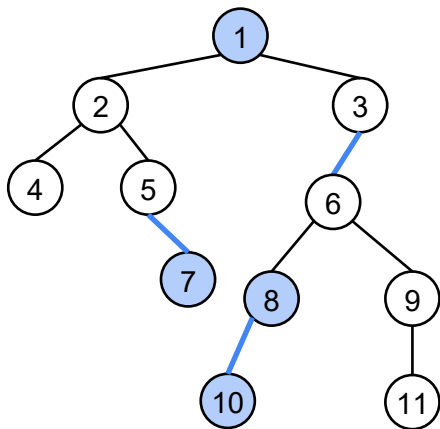
而此外的节点，仍参考 $k = 0$ 时的情况计算。



中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

当 k 没有限制，且根是中序遍历中确定的节点时。

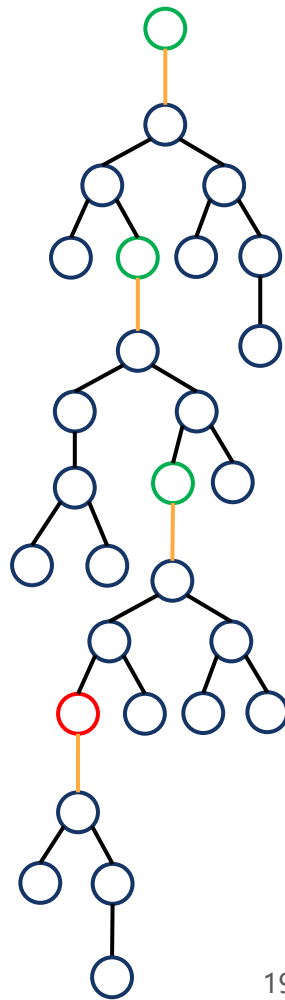
在 $k = 2$ 的基础上，我们再逐一考察节点 x 在中序遍历中的后一个节点 y 、再后一个节点 z 、.....我们发现每次多确定一个节点，这个节点的所有祖先的子节点的情况（即：如果只有一个子节点，那么到底是左儿子还是右儿子）就唯一确定了。



中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

当 $k = 1$ 时。

对于唯一确定的节点X（例如红色的那个节点），只有它和它的所有祖先的子节点的情况（即黄色的“垂直的竖线”往左摆还是往右摆）对它在中序遍历中的位置有影响，其它节点的子节点的情况对此毫无影响力。

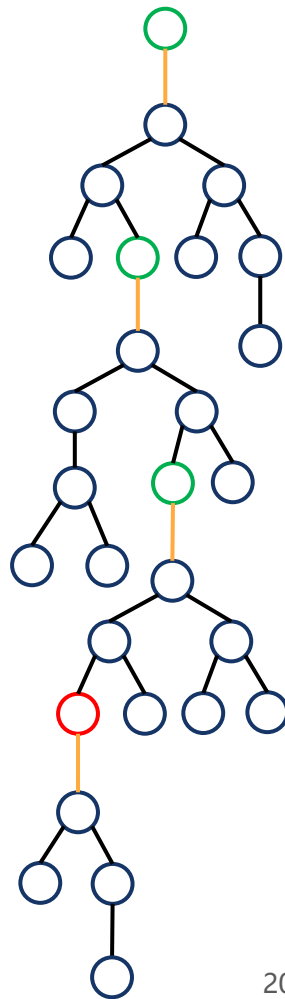


中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

首先，我们假设黄色的“垂直的竖线”都往左摆，那么可以计算出一个节点 x 在中序遍历中的位置。

然后，我们把部分“垂直的竖线”改为往右摆：

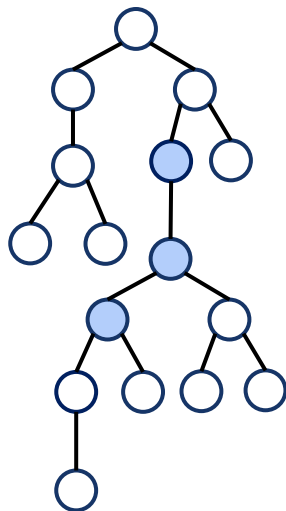
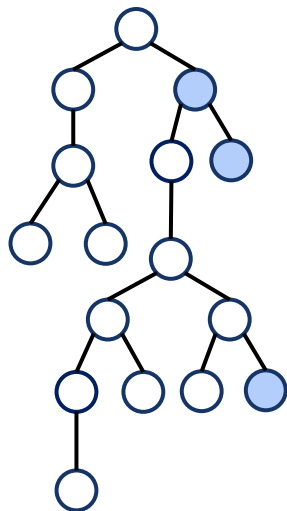
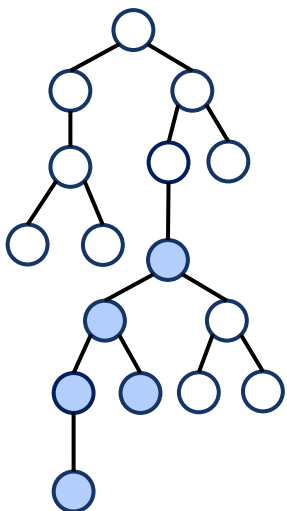
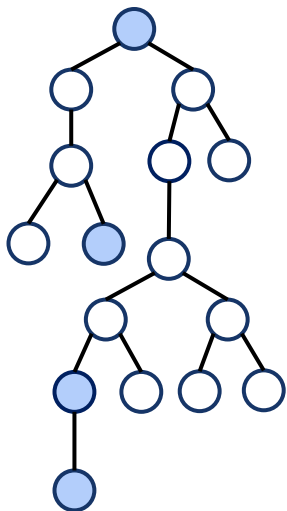
- 对于节点 X 所有祖先中只有一个子节点的节点（即绿色节点）如果改为往右摆后，节点 X 在中序遍历中的位置增加1。
- 对于节点 X 自身，如果它只有一个子节点，改为往右摆后，节点 X 在中序遍历中的位置减少 $(s - 1)$ ，其中 s 表示以节点 X 为根的子树的节点个数。



中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

当没有额外限制时。

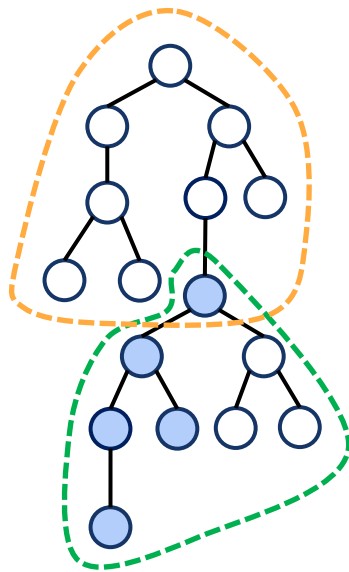
中序遍历中连续的一段节点，必有唯一的一个层数最小的节点 x （根节点的层数为0，叶节点的层数是树的深度）。所有确定的节点一定都在以节点 x 为根的子树里。



中序遍历 (ICPC西南欧赛区 SWERC 2023/2024)

如果节点 x 没有子节点 (则一定有 $k = 1$) 或有两个子节点, 那么: 它的祖先的子节点的情况, 和子树里的节点的子节点的情况, 是独立的。我们只要用 “当 $k = 1$ 时” 的方法计算它的祖先的情况个数, 用 “当 k 没有限制, 且根是中序遍历中确定的节点时” 的方法计算它的子树里的情况个数, 相乘即可。

如果节点 x 恰有一个子节点且 $k \geq 2$, 那么也可以如上所述计算。只不过, “当 $k = 1$ 时” 中的 “分情况讨论节点 x 自身的子节点往左摆还是往右摆的两种情况” 无需再讨论, 是唯一确定的。



GPUs (保加利亚第40届全国信息学奥林匹克, 2024)

现有 N 个任务, 每个任务都需要1张GPU跑1秒。其中, 第 i 个任务最早只能从第 T_i 秒开始运行。你有 M 张GPU, 其中第 j 张GPU跑1秒的单价是 C_j 。

你需要给每个任务指定在哪一秒、用哪张GPU跑。一张GPU在同一秒内只能跑一个任务。

完工时间 F 为最后一个被安排的任务的开始时刻+1。总花费 S 为每个任务用的GPU的花费之和。求 $F \times S$ 的最小值。

$N \leq 10^7$, $T_i, M \leq N$, $C_j \leq 2N$, 每个测试点包含最多5组数据。输入数据中的 T_i 和 C_j 都是按不降序给出的。时限1.4秒。

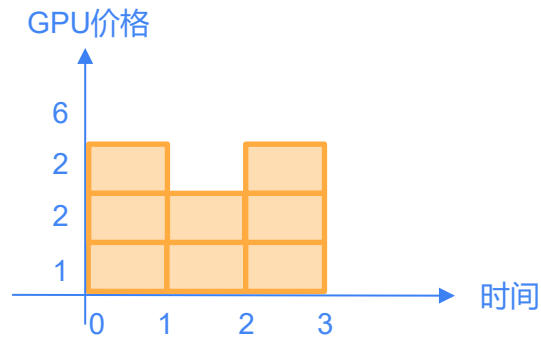
<https://infos.infosbg.com/contests/NOI/2024?lang=en>, 题目AB3_gpus

GPUs (保加利亚第40届全国信息学奥林匹克, 2024)

样例输入	样例输出
1 8 4 1 2 2 6 0 0 0 0 1 2 2 2	39

在本样例中, 有1组数据。该数据中有 $N = 8$ 个任务,
 $M = 4$ 张GPU。GPU的价格分别为1, 2, 2, 6, 任务的最早
开始时间分别为0, 0, 0, 0, 1, 2, 2, 2。

如果前 3 个任务在第 0 秒完成, 接下来的 2 个任务在第
1 秒完成, 最后 3 个任务在第 2 秒完成, 我们能做到总
花费 $S = 13$, 完工时间 $F = 3$ 。



GPUs (保加利亚第40届全国信息学奥林匹克, 2024)

设 U_t 表示在第 t 秒运行的任务数量。显然, 这些任务一定是在 U_t 个最便宜的GPU上运行的。

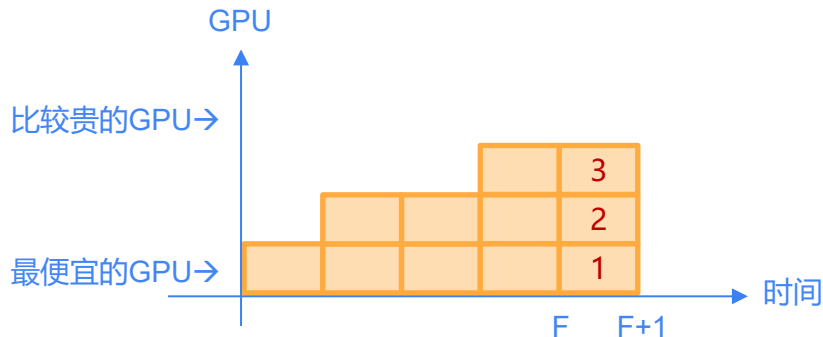
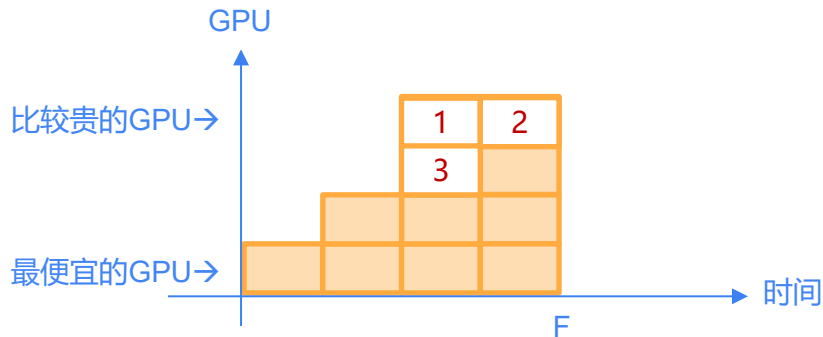
同时, 我们可以不妨设 $U_{t+1} \geq U_t$, 换句话说, 至少有一个最优解满足 $U_{t+1} \geq U_t$ 。否则, 我们只要不断地把一个任务从第 t 秒运行改为第 $t + 1$ 秒运行即可。

于是, 我们首先可以找到一个朴素的贪心算法。

首先, 从 $T_{max} + 1$ 开始, 从小到大逐一枚举完工时间 F 。然后按时间从晚到早处理每个任务, 将当前任务安排在最便宜的机器上, 并且在选定机器后安排尽可能晚的时间。处理完所有任务后计算出总成本 S , 并更新当前最小的 $F \times S$ 。

GPUs (保加利亚第40届全国信息学奥林匹克, 2024)

我们可以进一步优化这个贪心算法。由于我们是从小到大逐一枚举完工时间 F 的, 所以在处理 $F + 1$ 的任务安排情况时, 可以在 F 的任务安排的基础上调整。



如上图所示, 我们只要把一些任务从当前最贵的GPU中移出, 放到时间 $F + 1$ 上即可。

GPUs (保加利亚第40届全国信息学奥林匹克, 2024)

表面上看, 时间复杂度仍然为 $O(N^2)$, 但实际上是 $O(N \log N)$ 。

原因是, 总的时间复杂度等于总共需要做几次移动任务的操作。容易看出, 当完工时间从小到大枚举到 F 时, 至多需要移动 $N/(F - T_{max})$ 个任务。因此, 总的移动次数不超过:

$$N \times \left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{N}\right) = O(N \log N)$$

GPUs (保加利亚第40届全国信息学奥林匹克, 2024)

我们继续优化上述算法。刚开始当 $F = T_{max} + 1$ 时, 我们安排好任务后, 计算出此时每张GPU分配了多少任务, 记录在一个数组里。

当 F 增加1时, 我们首先把最后一秒排满, 即当前使用的所有 K 张GPU都增加一秒的工作。为此, 我们不会逐一更新每张GPU分配的任务数量, 而是用一个全局变量 $total_addition$ 记录增量。

然后我们再做减法, 由于容量增加了 K , 我们需要从最贵的那张GPU中减去 K 个任务。如果不够减, 则减到0后把它踢出, 并继续扣减下一张最贵的GPU, 以此类推。

由于每张GPU最多被踢出一次, 因此总的时间复杂度为 $O(N)$ 。

忒修斯（中欧信息学奥林匹克 CEOI 2025）

某个迷宫是一个连通无向图，有 n 个节点和 m 条边，以及一个特殊节点 t 。目标是从某个节点 s 进入迷宫，最终移动到终点 t 。

首先，你需要写一个函数`label`，在知道无向图的构造、知道 t 、但不知道 s 的情况下，给无向图的每条边上贴上0或1的标签。该函数只会被调用一次。

其次，你需要写另一个函数`travel`，确定每次下一步移动到哪个节点。该函数的参数仅有节点个数 n 、当前所在节点 u （ $\neq t$ ）、 u 的所有邻居信息。每条邻居信息 (v, e) 表示当前节点 u 与节点 v 之间有一条标签为 e 的边。换句话说，不能看到完整的无向图，也不知道 s 和 t 的值。该函数的返回值是下一步移动到的节点。如果返回 t ，系统将终止程序。

忒修斯（中欧信息学奥林匹克 CEOI 2025）

注意：两个函数之间不能互相交流，travel函数的多次调用之间也不能互相交流。

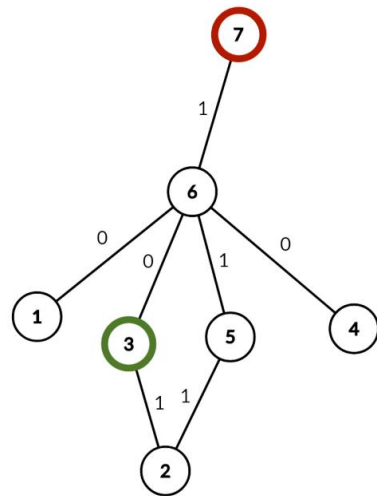
数据范围： $n \leq 10000$ ， $m \leq 50000$ ， $C = 14$ 。移动次数不得超过 $\min + C$ 次，其中 $\min$ 表示 s 到 t 最短路的长度。

<https://www.luogu.com.cn/problem/P13695>

忒修斯（中欧信息学奥林匹克 CEOI 2025）

样例交互：右图中起点 $s = 3$ ，终点 $t = 7$ 。首先用label函数确定了如下图所示的标签。然后对travel函数进行了下述调用，并当返回7时结束。

调用	返回值
<code>travel(7, 3, {{2, 1}}, {6, 0})</code>	2
<code>travel(7, 2, {{5, 1}}, {3, 1})</code>	5
<code>travel(7, 5, {{6, 1}}, {2, 1})</code>	6
<code>travel(7, 6, {{3, 0}}, {5, 1}, {1, 0}, {4, 0}, {7, 1})</code>	4
<code>travel(7, 4, {{6, 0}})</code>	6
<code>travel(7, 6, {{3, 0}}, {5, 1}, {1, 0}, {4, 0}, {7, 1})</code>	7



忒修斯（中欧信息学奥林匹克 CEOI 2025）

首先，给无向图中的每条边贴上0或1的编号，相当于把无向图变成有向图。因为我们要实现的两个函数都是能看到节点编号的，所以0或1的标签可以表示方向是“编号较小节点 $\rightarrow$ 编号较大节点”还是相反。

如果原图是一棵树（系第3个子问题），我们可以轻松地让每条边的方向都是“子节点 $\rightarrow$ 父节点”，那么travel函数只要让每一步都沿着有向边走即可。

这就提醒我们，可以先求 t 到所有其它节点的最短路，并按最短路长度给所有节点分层，终点 t 为第0层。

忒修斯（中欧信息学奥林匹克 CEOI 2025）

于是，如果一条边的两个节点位于不同层，我们可以让“较高层的节点→较低层的节点”，按这个方向走肯定是越来越靠近 t 的。当然，一般的图也肯定有同一层里的两个节点构成的边，我们下面将思考如何确定他们的方向。

我们非常希望每次调用`travel`函数时，都尽量走跨层的“高效”边，尽量不走两个节点在同一层中的“低效”边。根据题中的要求，“低效”边最多走 $C = 14$ 次，也就是 $\log n$ 次。

这就要求有一个权值，每走一次这种“低效”边，权值就翻倍，且最终最多不超过 n ，才能把次数控制在 $\log n$ 内。

忒修斯（中欧信息学奥林匹克 CEOI 2025）

我们可以给每个点都安排一个权值 f ，并在约定每条边的方向如下：

- 每条“高效”边约定为“较高层的节点 $\rightarrow$ 较低层的节点”
- 每条“低效”边约定为“ f 值较小的节点 $\rightarrow$ f 值较大的节点”

在确定好一条边的方向 $u \rightarrow v$ 后，令 $f_v += f_u$ ， $f_u = 0$ ，即把 u 的权值送给 v 。这样

- 当走过一条“高效”边 $u \rightarrow v$ 时，能保证 $f_v \geq f_u$
- 当走过一条“低效”边 $u \rightarrow v$ 时，能保证 $f_v \geq 2f_u$

（上述提到的 f 值，指的是变成0之前的值）

经过这样的安排，我们就能保证在非最短路径上最多浪费 $\log n$ 步。

忒修斯（中欧信息学奥林匹克 CEOI 2025）

细节方面，我们可以先对所有的边 (u, v) （其中 $u < v$ ）按 $\max(\text{层数}_u, \text{层数}_v)$ 排序，如有并列则按 u ，仍有并列则按 v ，每个关键字均为从大到小排序。我们按此顺序逐一处理每一条边。

初始时每个节点的权值 f 均为 1。按上一页中提到的方法决定每条边的方向后，每个节点送出权值的对象一定是它的出边中，终点编号最大的那一个。

在实现 `travel` 函数时，只要规定若某个节点有多个出边，则选终点编号最大的那个。这样，就能沿着权值 f 不断递增的路走，从而达到上一页中的步数期待。

植物法典（意大利信息学奥林匹克 OII 2024）

给定一棵 N 个节点的树（ $N \leq 300\,000$ ），节点编号为 $0 \sim N - 1$ ，其中节点 0 为根。每个节点的高度是指从根节点到它的距离。

你可以删除其中的一些节点，使得剩余的节点仍然构成一棵连通的树。

你的目标是：

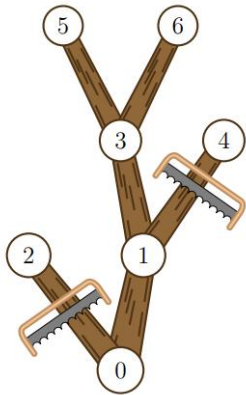
- 保证剩下的树中的叶子节点的高度全部相同
- 删除的节点个数尽可能少

输入 N 和每个节点的父节点编号（约定根节点的父节点为 -1 ），输出最少删除多少个节点。

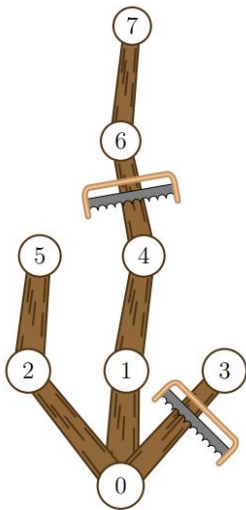
https://training.olinfo.it/task/oii_botanicus

植物法典（意大利信息学奥林匹克 OII 2024）

样例输入	样例输出
7 -1 0 0 1 1 3 3	2



样例输入	样例输出
8 -1 0 0 0 1 2 4 6	3



植物法典（意大利信息学奥林匹克 OII 2024）

题目里问的是“砍掉哪些边”，但其实我们可以换一个思路，考虑“保留哪些节点”。

当最终高度为 h 时，哪些点可以保留呢？

- 该节点本身的高度 $\leq h$
- 该节点必须有一个子孙节点，其高度 $\geq h$

因此，我们可以对每个节点都做一次预处理，计算该节点自身的高度（记为 a ），以及以该节点为根的子树最深的子孙的高度（记为 b ）。

植物法典（意大利信息学奥林匹克 OII 2024）

现在问题变成了：我有 N 个区间（每个节点对应一个 $[a, b]$ ），哪个整数 h 被最多的区间覆盖了？

这就是一个经典的差分数组与前缀和问题。

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

游乐园里有一个过山车项目。过山车从房间0出发，经过房间1,2,...,最终到房间 $N - 1$ 。共有 N 个开关，每个房间都对应一个，但你不知道开关和房间的对应关系。

你的目标是打开房间 0、房间 $N - 1$ 的灯（不计顺序），并关闭其余所有房间的灯。

为此，你可以选择性地打开部分开关，然后启动过山车做一次实验。每当过山车从开灯的房间进入关灯的房间，或者从关灯的房间进入开灯的房间，游客都会尖叫一次。于是，你可以记下游客尖叫的次数（尖叫的时间没有意义，因为过山车的速度是不确定的）。

$N \leq 30\,000$ ，你最多可以做30次实验。

<https://www.luogu.com.cn/problem/P13340>

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

系统输出	用户输出
5	
	? 10001
3	
	? 10110
3	
	! 2 4

在本样例中，5个开关对应的房间号分别为2, 1, 0, 3, 4。

首先，系统输出5，表示有5个房间。

然后，用户打开第0, 4个开关，它们对应的是房间2, 4，系统输出3次尖叫（进入房间2, 3, 4时各尖叫一次）。

然后，用户打开第0, 2, 3个开关，它们对应的是房间2, 0, 3，系统仍然输出3次尖叫（进入房间1, 2, 4时各尖叫一次）。

最后，用户输出! 2 4，表示第2, 4个开关（不计顺序）对应着第0、第4（即 $N - 1$ ）个房间。

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

系统输出	用户输出
3	
	? 111
0	
	? 110
2	
	? 000
0	
	! 1 0

系统输出	用户输出
4	
	? 1010
3	
	! 0 3

黑暗之旅 (欧洲女生信息学奥林匹克 EGOI 2025)

子问题如下:

1. $N = 3$
2. $N \leq 30$
3. 开关0控制房间0
4. N 为偶数, 且两头房间 (房间0、房间 $N - 1$) 的开关中恰好有一个在前半段 $[0, N/2)$, 另一个在后半段 $[N/2, N)$
5. $N \leq 1000$
6. 无额外限制条件

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

子问题1、2较为简单，不再赘述。

对于子问题3，由于已知开关0控制房间0，我们可以总是关闭开关0，让房间0总是关灯状态。

那么，当另一头的房间 $N - 1$ 也是关灯状态时，尖叫次数一定是偶数；而房间 $N - 1$ 是开灯状态时，尖叫次数一定是奇数。其它任何房间的状态都不会影响尖叫次数的奇偶性。

由此结论，我们可以对除了开关0以外的所有开关使用二分法，来查找哪个开关控制房间 $N - 1$ 。

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

下面，为了描述简单起见，我们称控制房间0或 $N - 1$ 的开关为“重要开关”。

对于子问题4，我们可以用控制变量法来向上一个子问题靠拢。

由于知道重要开关一定有一个在左半边，那么我们直接安排左半边的所有开关为关灯状态。这样，在左半边的重要开关就一定处于关灯状态，尖叫次数的奇偶性就仅取决于右半边的重要开关的开关的状态。

于是，我们可以对右半边的开关用二分法，找到哪个是重要开关。

同理，我们可以再安排右半边的所有开关为关灯状态，用二分法找到左半边的开关里哪个是重要开关。

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

对于子问题5，我们也尽量向上一个子问题靠拢。在本子问题中，只要能把所有开关分成两部分，使得每一部分里恰好有一个重要开关，就把问题归化为了子问题4。

首先，我们可以把开关分为左边和右边两部分，只打开左边部分的所有开关。如果此时的尖叫次数为奇数，那么一定是每一部分里都恰有一个重要开关，问题就解决了。

如果此时的尖叫次数为偶数，我们只知道两个重要开关一定在同一部分，但不知道到底在哪部分。我们不管在哪部分，直接同步二分，把每一部分再分为左右两个部分，这样从左至右一共分为4个小部分，我们打开第1小部分+第3小部分，看尖叫次数的奇偶性。如果是奇数，那么问题解决了。

如果仍然为偶数，就继续把每一小部分再分，一共分为8个小部分，以此类推。

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

最后，当每一小部分只有1个开关时，一定会遇到一次尖叫次数为奇数。从而，我们可以归纳到上一个子问题。

对于上述过程，我们换一个思路，也可以这么理解：两个重要开关的二进制表示中，总有一位是不同的。我们上述这些问题就是逐一询问二进制的每一位，看哪一位不同。

子问题4需要询问的次数为 $2 \log N$ ，本子问题归纳到子问题4之前需要询问的次数为 $\log N$ ，总共不超过 $3 \log N$ ，当 $N \leq 1000$ 时可以在30次询问内完成。

黑暗之旅（欧洲女生信息学奥林匹克 EGOI 2025）

对于子问题6，我们可以受到前面子问题的启发，继续优化查询次数。

在子问题5中，我们一旦遇到一次是奇数的尖叫次数就停下来，然后归纳到子问题4的解法。换句话说，我们一旦找到两个重要开关的二进制里哪一位不同就停下来了，不再比较了。

其实，我们可以无论结果如何，都继续比较下去，从而得知它们的每一位二进制是相同还是不同。这样做的好处是，只要知道了其中一个重要开关的二进制表示，另一个重要开关也随之一位一位地确定。

然后，我们归纳到子问题4的解法，不过只做一半就够了：找出其中一个重要开关的位置。接着根据上面的方法，确定另一个重要开关。总共仅需 $2 \log N$ 次比较。

展览（瑞士信息学奥林匹克 SOI 2023/2024 第一轮）

有 K 位画家，每位都有若干幅作品，加起来一共有 N 幅。每幅作品都有一定的价值。

对于所有的 $2 \leq i \leq N$ ，请计算恰好选取 i 副作品时，其价值之和的最大值。要求在选取 i 副作品时，每位画家的作品要么不选，要么至少选2幅。

$N \leq 80\,000$ ， $K \leq 40\,000$ ，每幅作品的价值 $\leq 100\,000$ 。

<https://soi.ch/contests/2024/round1/exhibition/>

题解：https://soi.ch/media/files/24r1_en.pdf，3~6页

展览（瑞士信息学奥林匹克 SOI 2023/2024 第一轮）

样例输入	样例输出
3 9 3 8 2 4 2 7 3 4 9 1 3 5	14 17 26 29 36 39 41 42

本样例中，有3位画家，共9幅作品。其中第1位画家有3幅作品，它们的价值分别是8, 2, 4；第2位画家有2幅作品，价值分别位7, 3；等等。

选取2幅作品时，价值之和最大是14，即选取第3位画家的第1, 4幅作品。

选取3幅作品时，价值之和最大是17，即选取第3位画家的第1, 3, 4幅作品。

选取4幅作品时，价值之和最大是26，即选取第1位画家的第1, 3幅作品，第3位画家的第1, 4幅作品。

展览（瑞士信息学奥林匹克 SOI 2023/2024 第一轮）

当画家数量较少时，可以用动态规划求解。

首先对每个画家，显然会优先考虑其价值较高的作品。因此可以按其作品的价值降序排列，预处理前缀和。

设 $f[i][j]$ 表示在前 i 个画家的范围内，选 j 幅画能达到的最大总价值，如果无法选 j 幅画则为 -1 。

当我们要从 $f[i][*]$ 转移到 $f[i + 1][*]$ 时，只要枚举第 $i + 1$ 个画家选取了几幅作品，注意不能是1幅即可。

另外，我们可以使用滚动数组而不需要二维数组。时间复杂度为 $O(NK)$ 。

展览（瑞士信息学奥林匹克 SOI 2023/2024 第一轮）

当画家数量较多时，我们考虑一种贪心法。

由于每位画家最贵的2幅作品要么都取、要么都不取，因此我们想到可以把它们俩组成一个“对子”，并把其中每个的价值都改为对子总价值的一半。同时，把它不是对子的作品都称为“单张”。

如此操作后，我们从大到小对所有作品按价值排序。价值相同的按画家序号排（升序降序都行）。画家序号仍然相同的，对子排前面，单张排后面。

这时，如果需要 i 幅作品，且选取上述排列的前 i 个元素时，没有把对子拆开，显然是满足要求的最大值。因为根据排序规则，任何一个画家的对子的价值一定不低于其单张的价值，且对子排在前面、又没有被拆开。不可能有其它总价值更高的选取方法了。

展览（瑞士信息学奥林匹克 SOI 2023/2024 第一轮）

|AA| |A| |BB| |C|C| |DD| |D| |B| |A|

麻烦在于，如果需要 i 幅作品，且选取上述排列的前 i 个元素时，把对子拆开的情况。可以证明，只需要选择下面三种补救方法中的一种：

1. 放弃被拆开的对子。向后找到第一个单张，并补充进去。例如：

|AA| |A| |BB| |C|C| |DD| |D| |B| |A|

2. 保留被拆开的对子。向前找到第一个单张，并放弃。例如：

|AA| |A| |BB| |C|C| |DD| |D| |B| |A|

3. 当被拆开的对子之前也紧接着一个对子时，放弃这两个对子。向后找同一画家的前三副作品里价值最大的。例如：

|AA| |A| |BB| |C|C| |DD| |D| |B| |A|

上述贪心法的正确性如何证明？

展览（瑞士信息学奥林匹克 SOI 2023/2024 第一轮）

|AA| |A| |BB| |C|C| |DD| |D| |B| |A|

我们可以枚举在上述排列的前 $i - 1$ 个元素中，我们需要放弃多少个。

1. 放弃0个。显然在后面的元素中，补充价值最大的单张，即为补救方法1。
2. 放弃1个。显然在前 $i - 1$ 个元素中放弃价值最小的单张，后面的元素中补充价值最大的2个元素（一定为被拆开的对子），即为补救方法2。
3. 放弃2个。下面证明必须是补救方法3。
 - 必须在后面的元素中找到价值最大的3个元素。但凡找的不是某个画家的前3幅作品，而是3个单张，或出自两个画家，一定可以和放弃的2个元素置换其中的1个或2个，而成为更优解。
 - 必须放弃前 $i - 1$ 个元素中的最后2个元素，且它们必须是对子。如果它们是2个单张，则可以和后面的某个单张置换，成为更优解。如果它们不是最后2个元素，则可以改为不放弃它们，放弃它们后面的1个单张，另外在第 i 个元素后面少补充1个单张，而成为更优解。
4. 放弃3个或以上。可以证明，不可能为唯一的最优解。

让一切相同（西欧信息学奥林匹克 WEOI 2024）

有 N 堆石子，其中 $N = 2^k$ ，是2的幂次。每堆石子的高度是 H_i 。石子会按高度从矮到高排列： $H_0 \leq H_1 \leq \dots \leq H_{N-1}$ 。

你不知道每堆石子的高度，但可以做两种操作：

- $add(S, X)$ ：指定一个正整数 X ，以及 N 堆（排完序的）石子的一个子集 S 。系统会将 S 中的每堆石子都增高 X 。然后，立即重新排序，使得它们仍然是从矮到高排列： $H_0 \leq H_1 \leq \dots \leq H_{N-1}$ 。当然，同一堆石子的下标在排序后可能会变化。
- $compare(i, j)$ ：询问当前第 i 堆和第 j 堆石子是否一样高，即 H_i 是否等于 H_j 。

<https://codeforces.com/gym/105259/problem/A>

让一切相同（西欧信息学奥林匹克 WEOI 2024）

本题有多个子问题：

- 5分： $N = 2, H_i \leq 4$ ， *add*操作3000次， *compare*操作4000次
- 16分： $N = 2, H_i \leq 1\,000\,000$ ， *add*操作22次， *compare*操作1次
- 15分： $N = 256, H_i \leq 10$ ， *add*操作3000次， *compare*操作255次
- 18分： $N = 4, H_i \leq 1\,000\,000$ ， *add*操作45次， *compare*操作255次
- 22分： $N = 2048, H_i \leq 1\,000\,000$ ， *add*操作298次， *compare*操作400次， 且保证初始时最多只有两种不同的高度
- 24分： $N = 2048, H_i \leq 1\,000\,000$ ， *add*操作298次， *compare*操作2047次

让一切相同（西欧信息学奥林匹克 WEOI 2024）

样例： $N = 4$ 堆石子，高度分别为1, 4, 5, 5。注意：任何时刻你都看不到石子高度。

操作	返回值	石子高度	解释
compare(1,2)	false		$H_1 = 4, H_2 = 5$ ，因此 $H_1 \neq H_2$
compare(2,3)	true		$H_2 = 5, H_3 = 5$ ，因此 $H_2 = H_3$
add({0,1},2)		3, 5, 5, 6	add操作后，新的石子高度为3, 6, 5, 5，排序后即3, 5, 5, 6
compare(1,2)	true		$H_1 = 5, H_2 = 5$ ，因此 $H_1 = H_2$
add({0,3},3)		5, 5, 6, 9	add操作后，新的石子高度为6, 5, 5, 9，排序后即5, 5, 6, 9
add({0,1},4)		6, 9, 9, 9	add操作后，新的石子高度为9, 9, 6, 9，排序后即6, 9, 9, 9
add({0},3)		9, 9, 9, 9	add操作后，新的石子高度为9, 9, 9, 9，排序后即9, 9, 9, 9

让一切相同（西欧信息学奥林匹克 WEOI 2024）

首先考虑 $N = 2$ 的情况，看看有什么启发。

由于我们只知道 $H_i \leq 1\,000\,000 \leq 2^{20}$ ，因此我们最多知道 $H_1 - H_0 \leq 2^{20}$ 。于是，我们可以做 $add(\{0\}, 2^{19})$ 操作，这样无论 $H_1 - H_0$ 的值是多少，都能让操作后重新排序后的 $H_1 - H_0 \leq 2^{19}$ 。

以此类推，我们继续依次做 $add(\{0\}, 2^i)$ 操作（ $i = 18, 17, \dots, 0$ ），就能最终让 H_1 和 H_0 的差 ≤ 1 。

此时，我们用一次 $compare(0, 1)$ 操作，若相等就结束，否则再进行一次 $add(\{0\}, 1)$ 操作就大功告成了。

让一切相同（西欧信息学奥林匹克 WEOI 2024）

当 N 较大时，我们可以先通过类似的一轮操作，让第 $2k$ 项和第 $2k + 1$ 项都相等
($k = 0, 1, \dots, \frac{N}{2} - 1$)。

具体操作模式是：依次做 $add(\{0, 2, 4, \dots, N - 2\}, 2^i)$ 操作 ($i = 19, 18, \dots, 0$)，就能
最终让 H_{2k+1} 和 H_{2k} 的差 ≤ 1 ($k = 0, 1, \dots, \frac{N}{2} - 1$)。

此后，我们用 $\frac{N}{2}$ 次 $compare(2k, 2k + 1)$ 操作 ($k = 0, 1, \dots, \frac{N}{2} - 1$)，每次 $compare$
的结果若相等就继续，不相等就进行一次 $add(\{2k\}, 1)$ 。

让一切相同 (西欧信息学奥林匹克 WEOI 2024)

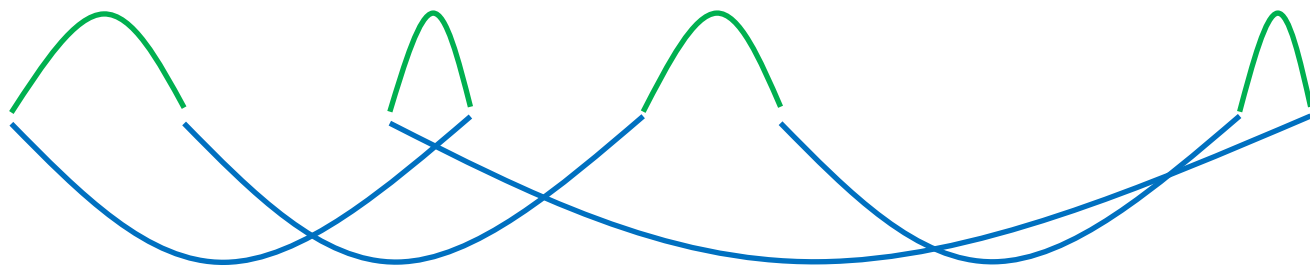
一个例子:

操作	石子高度
初始	1, 4, 6, 6, 7, 10, 12, 13
add($\{0, 2, 4, 6\}, 8$)	9, 4, 14, 6, 15, 10, 20, 13
排序后	4, 6, 9, 10, 13, 14, 15, 20
add($\{0, 2, 4, 6\}, 4$)	8, 6, 13, 10, 17, 14, 19, 20
排序后	6, 8, 10, 13, 14, 17, 19, 20
add($\{0, 2, 4, 6\}, 2$)	8, 8, 12, 13, 16, 17, 21, 20
排序后	8, 8, 12, 13, 16, 17, 20, 21
add($\{0, 2, 4, 6\}, 1$)	9, 8, 13, 13, 17, 17, 21, 21
排序后	8, 9, 13, 13, 17, 17, 21, 21

让一切相同 (西欧信息学奥林匹克 WEOI 2024)

那么, 我们是如何确保上述方法的正确性的? 换句话说, 如果当前 H_{2k+1} 和 H_{2k} 的差 $\leq 2^{i+1}$ ($k = 0, 1, \dots, \frac{N}{2} - 1$), 我们如何保证做了 $add(\{0, 2, 4, \dots, N - 2\}, 2^i)$ 操作并重新排序后, 这个差 $\leq 2^i$?

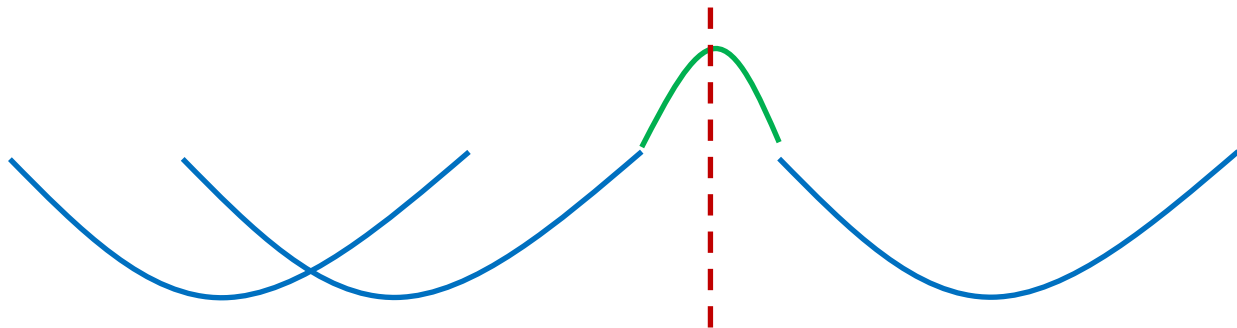
根据 $N = 2$ 时的思路, 我们可以确定做了 add 操作但尚未重新排序时, 这个差 $\leq 2^i$ 。下面要证明, 重新排序后, 这个结论仍然正确。



让一切相同（西欧信息学奥林匹克 WEOI 2024）

我们设蓝色的线段表示一对 add 操作后尚未排序时的 H_{2k} 和 H_{2k+1} ，绿色线段表示一对重新排序后的 H_{2k} 和 H_{2k+1} 。

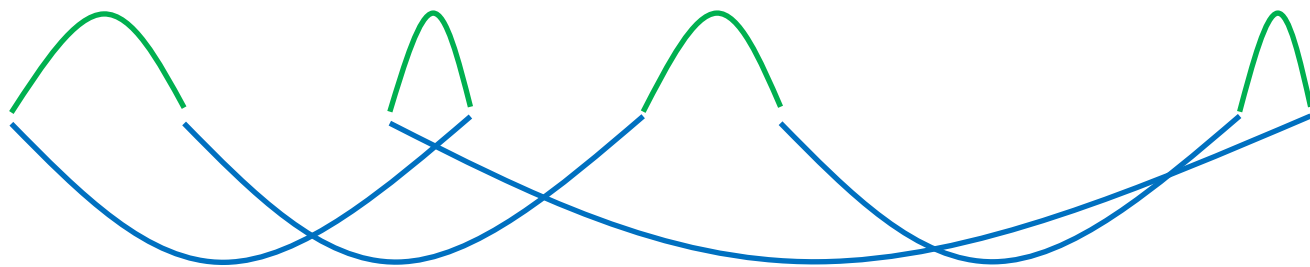
由于进行了重新排序，每一条绿色线段内部不存在任何一堆其它石子。如果在一条绿色线段内部把所有石子割成两半，那么不可能出现下面的这种没有割到任何蓝色线段的情况。否则，绿色线段的左端点下标一定为奇数，矛盾。



让一切相同（西欧信息学奥林匹克 WEOI 2024）

因此，任何一条绿色线段一定包含于一条蓝色线段内。

而蓝色线段的长度 $\leq 2^i$ ，于是绿色线段的长度也一定 $\leq 2^i$ 。



让一切相同 (西欧信息学奥林匹克 WEOI 2024)

通过上述的一轮操作，我们就得到了诸如9,9,13,13,17,17,21,21这样的序列。

然后，在第2轮操作中，把上一轮结束后的 H_{2k} 和 H_{2k+1} 捆绑起来，所有的 add 操作中 H_{2k} 和 H_{2k+1} 都会同步操作，这样就相当于一个长度为 $\frac{N}{2}$ 的序列，回到了第1轮操作的情况。完成这一轮操作，会得到诸如13,13,13,13,21,21,21,21这样的序列。

最后，通过约 $\log N$ 轮操作，所有的石子都将变成相同的高度。

导航（欧洲初中生信息学奥林匹克 EJOI 2025）

现有一个无向连通的简单仙人掌图（无向、连通、无自环、无重边、每条边至多属于一个简单环），有 N 个节点（ $N \leq 1000$ ）、 M 条边。每个节点有一个颜色（ $0 \sim 1499$ 的整数），初始颜色均为0。

有一个确定性的、无记忆的机器人，从某个节点开始遍历该图。它每一步只能看到当前节点的颜色，以及所有邻居节点（顺序固定）的颜色。根据这些颜色信息，机器人可以做以下两件事之一：

- 终止遍历
- 为当前节点选择一个新颜色（可以不变），并移动到它的一个邻居

机器人“无记忆”是指不能存储历史状态，只能根据当前颜色信息做确定性的决策。

<https://www.luogu.com.cn/problem/P14002>

导航（欧洲初中生信息学奥林匹克 EJOI 2025）

请设计一个机器人的决策算法，让机器人在最多3000步内访问所有节点至少一次，并自行终止。

你需要实现下面的函数：

```
std::pair<int,int> navigate(int currColor, std::vector<int>  
adjColors)
```

其中currColor表示当前节点的颜色、adjColors表示邻居节点列表；返回值的第一个元素表示当前节点的新颜色，第二个元素表示移动到哪里，用邻居节点列表里的下标表示。

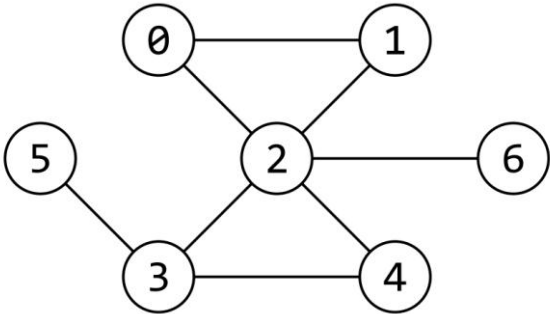
此外，本题的分数与使用的颜色数量有关，最多用4种颜色才能得到满分。

导航（欧洲初中生信息学奥林匹克 EJOI 2025）

在下述该样例中，我们用了6个不同的颜色：
0, 1, 2, 3, 4, 8，在9次操作后终止。这是一次失败的交互，因为我们从未访问到节点1。

#	节点颜色	节点	调用 navigate	返回值
1	0,0,0,0,0,0,0	5	navigate(0,{0})	{1,0}
2	0,0,0,0,0,1,0	3	navigate(0,{0,1,0})	{4,2}
3	0,0,0,4,0,1,0	2	navigate(0,{0,4,0,0,0})	{0,3}
4	0,0,0,4,0,1,0	6	navigate(0,{0})	{1,0}
5	0,0,0,4,0,1,1	2	navigate(0,{0,4,0,1,0})	{8,0}
6	0,0,8,4,0,1,1	0	navigate(0,{8,0})	{3,0}
7	3,0,8,4,0,1,1	2	navigate(8,{3,4,0,1,0})	{2,2}
8	3,0,2,4,0,1,1	4	navigate(0,{2,4})	{1,1}
9	3,0,2,4,1,1,1	3	navigate(4,{1,1,2})	{-1,-1}

节点	邻居顺序
0	2,1
1	2,0
2	0,3,4,6,1
3	4,5,2
4	2,3
5	3
6	2

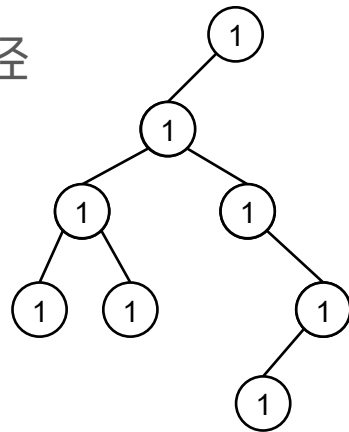


导航（欧洲初中生信息学奥林匹克 EJOI 2025）

首先，我们先考虑一种特殊情况，即原图是一棵树。

我们可以仅用3种颜色：INIT(0)表示尚未访问过，DONE(1)表示已经彻底访问完毕，DFS(2)表示仍在DFS过程中。

- 如果当前节点是INIT(0)
 - 如果周围还有INIT(0)，就继续走向INIT(0)，并把自己染为DFS(2)
 - 如果周围没有INIT(0)，就走向DFS(2)，并把自己染为DONE(1)
- 如果当前节点是DFS(2)
 - 如果周围还有INIT(0)，就继续走向INIT(0)
 - 如果周围没有INIT(0)，还有DFS(2)，就走向DFS(2)，并把自己染为DONE(1)
 - 如果周围没有INIT(0)、没有DFS(2)，就终止遍历



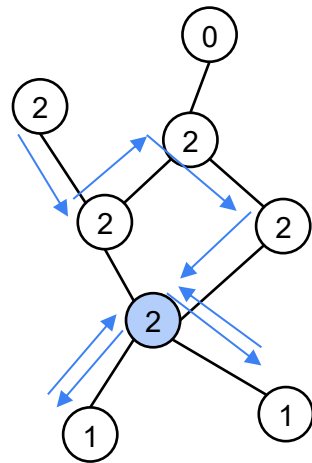
右图中的动画展示了一个具体的访问和染色的过程。

导航（欧洲初中生信息学奥林匹克 EJOI 2025）

然而，当原图是普通的仙人掌图时，如果仿照刚刚树结构时的染色方案，由于存在环，我们可能会在某些节点时，碰到本身是DFS(2)，而周围有超过1个DFS(2)的邻居，而不知道应该往哪里走的情况。

此时，我们引入第4种颜色PAR(3)，用于启动“找父节点”的流程。

- 当当前节点没有 INIT(0) 邻居，但看到2个DFS(2)/PAR(3)时，说明这个点处在环附近：一个是父节点，一个是回边指向的祖先。我们把当前点染成 PAR(3)，表示“我正在找父节点”，并先去试探其中一个候选节点。
- 接下来，我们需要一个办法，让被试探的那个候选“表态”：它到底是不是父节点。这个表态也只能靠颜色完成。

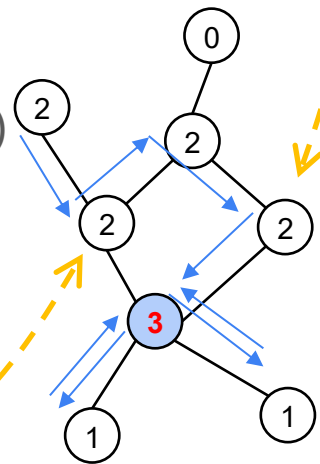


导航（欧洲初中生信息学奥林匹克 EJOI 2025）

在仙人掌图里，一个点最多只会遇到“一次回边歧义”，并且歧义候选最多只有两个。因此，我们可以用PAR(3)做一次很短的“问路”。

当某个点在DFS(2)状态下，看到邻居里有PAR(3)，可以把这个PAR(3)邻居理解为：它在问“你是不是我的父节点？”我们只需要看一眼：除去问我的那个邻居之外，我身边还有多少DFS(2)/PAR(3)：

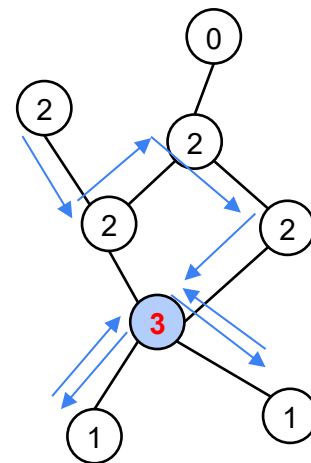
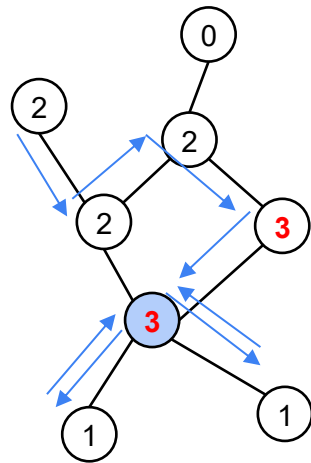
- 如果剩下 ≤ 1 个
 - 我像是“路径端点”，即父节点
 - 我把自己也染成PAR(3)（表示“我就是父节点”），然后走回去
- 如果剩下 ≥ 2 个
 - 我像是“路径中间”，即回祖边
 - 我保持DFS(2)，走回去（等它换另一个候选节点）



导航 (欧洲初中生信息学奥林匹克 EJOI 2025)

PAR(3) 节点收到回复后怎么做:

- 如果它现在看到邻居里出现了另一个 PAR(3), 说明父节点已确认。它把自己染成DONE(1), 回溯到那个PAR(3)邻居
- 否则说明刚才试探失败, 去试探另一个DFS(2)候选节点

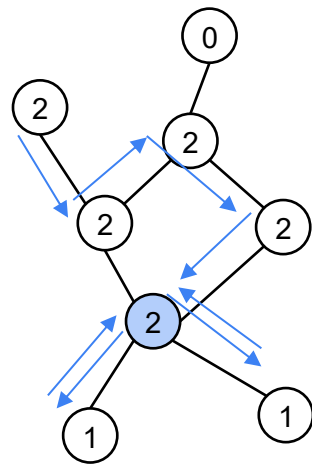


导航（欧洲初中生信息学奥林匹克 EJOI 2025）

总的来说，整体策略为：

- 能走INIT(0)就走INIT(0)
- 没有INIT(0)就回溯：若回溯方向唯一，直接DONE(1)并回去
- 回溯方向不唯一就问路：染PAR(3)启动试探，最多试两次就能确认父节点，然后继续回溯

我们总是在做DFS，只是当遇到“环造成的父节点歧义”时，用PAR(3)进行一次局部、可终止的判定。仙人掌结构保证这个判定不会无限扩散，也不会出现三个以上候选。



小结

小结

- 包括欧洲各地信息学赛事在内的许多国际比赛有不少形式新颖、思维方面要求较高、风格各异的题目。这些题目和解题方法，可供中国选手参加国际比赛前练习和参考，也可为拓展NOI系列活动的科技树提供新的启发。
- 部分欧洲国家在人口基数少的情况下，仍然产生了一定数量的优秀选手，他们的培养模式和各轮、各阶段的题目值得我们参考和取长补短。
- 最后，感谢同学们参与本次冬令营课程，并欢迎提出你们的想法和建议：)